

Workflow and architecture patterns for working together 2: Architectures

Jacob Archambault

May 22, 2026

Outline

1 Current challenges and their canonical solutions

Outline

- 1 Current challenges and their canonical solutions
- 2 Going forward
 - Team Composition
 - Committing
 - Branching
 - Pull Requests
 - Pipelines
 - Architecture

Challenges

- multiple teams working in the same repository

Challenges

- multiple teams working in the same repository
- lack of direct collaboration, and occasionally trust, between development, operations, and testing

Challenges

- multiple teams working in the same repository
- lack of direct collaboration, and occasionally trust, between development, operations, and testing
- late integration of contributor code

Challenges

- multiple teams working in the same repository
- lack of direct collaboration, and occasionally trust, between development, operations, and testing
- late integration of contributor code
 - feedback delays

Challenges

- multiple teams working in the same repository
- lack of direct collaboration, and occasionally trust, between development, operations, and testing
- late integration of contributor code
 - feedback delays
 - merge conflict regressions

Challenges

- multiple teams working in the same repository
- lack of direct collaboration, and occasionally trust, between development, operations, and testing
- late integration of contributor code
 - feedback delays
 - merge conflict regressions
 - sunk costs fallacy risks

Challenges

- multiple teams working in the same repository
- lack of direct collaboration, and occasionally trust, between development, operations, and testing
- late integration of contributor code
 - feedback delays
 - merge conflict regressions
 - sunk costs fallacy risks
- coupling of all services at the database layer

Challenges

- multiple teams working in the same repository
- lack of direct collaboration, and occasionally trust, between development, operations, and testing
- late integration of contributor code
 - feedback delays
 - merge conflict regressions
 - sunk costs fallacy risks
- coupling of all services at the database layer
 - high storage costs

Challenges

- multiple teams working in the same repository
- lack of direct collaboration, and occasionally trust, between development, operations, and testing
- late integration of contributor code
 - feedback delays
 - merge conflict regressions
 - sunk costs fallacy risks
- coupling of all services at the database layer
 - high storage costs
 - single point of failure risk

Challenges

- multiple teams working in the same repository
- lack of direct collaboration, and occasionally trust, between development, operations, and testing
- late integration of contributor code
 - feedback delays
 - merge conflict regressions
 - sunk costs fallacy risks
- coupling of all services at the database layer
 - high storage costs
 - single point of failure risk
 - risk of cross-project interference

Challenges

- multiple teams working in the same repository
- lack of direct collaboration, and occasionally trust, between development, operations, and testing
- late integration of contributor code
 - feedback delays
 - merge conflict regressions
 - sunk costs fallacy risks
- coupling of all services at the database layer
 - high storage costs
 - single point of failure risk
 - risk of cross-project interference

Canonical solutions to these challenges

- multiple teams $\rightarrow$ inverse Conway maneuver

Canonical solutions to these challenges

- multiple teams → inverse Conway maneuver
- collaboration → business stream-aligned teams over activity oriented teams

Canonical solutions to these challenges

- multiple teams → inverse Conway maneuver
- collaboration → business stream-aligned teams over activity oriented teams
- late integration → adopting a true continuous integration workflow

Canonical solutions to these challenges

- multiple teams → inverse Conway maneuver
- collaboration → business stream-aligned teams over activity oriented teams
- late integration → adopting a true continuous integration workflow
- database coupling → Bezos API mandate/“Microservices”

Canonical solutions to these challenges

- multiple teams → inverse Conway maneuver
- collaboration → business stream-aligned teams over activity oriented teams
- late integration → adopting a true continuous integration workflow
- database coupling → Bezos API mandate/“Microservices”

Outline

- 1 Current challenges and their canonical solutions
- 2 **Going forward**
 - **Team Composition**
 - Committing
 - Branching
 - Pull Requests
 - Pipelines
 - Architecture

Team composition

- all teams in same repo are in the same daily scrum

Team composition

- all teams in same repo are in the same daily scrum
- each team in the above sense has its own small Microsoft Teams channel with only its own contributors in it

Team composition

- all teams in same repo are in the same daily scrum
- each team in the above sense has its own small Microsoft Teams channel with only its own contributors in it
- these teams are *stream-aligned*, i.e., they focus on a particular business area

Team composition

- all teams in same repo are in the same daily scrum
- each team in the above sense has its own small Microsoft Teams channel with only its own contributors in it
- these teams are *stream-aligned*, i.e., they focus on a particular business area
- these teams are *autonomous*, i.e., they can move work forward without requiring outside assistance from another team

Team composition

- all teams in same repo are in the same daily scrum
- each team in the above sense has its own small Microsoft Teams channel with only its own contributors in it
- these teams are *stream-aligned*, i.e., they focus on a particular business area
- these teams are *autonomous*, i.e., they can move work forward without requiring outside assistance from another team

Outline

1 Current challenges and their canonical solutions

2 Going forward

- Team Composition
- **Committing**
- Branching
- Pull Requests
- Pipelines
- Architecture

- every commit gets a Jira story

Outline

1 Current challenges and their canonical solutions

2 Going forward

- Team Composition
- Committing
- **Branching**
- Pull Requests
- Pipelines
- Architecture

- team branch

Outline

1 Current challenges and their canonical solutions

2 Going forward

- Team Composition
- Committing
- Branching
- **Pull Requests**
- Pipelines
- Architecture

- daily PRs - stacked PRs

Outline

1 Current challenges and their canonical solutions

2 Going forward

- Team Composition
- Committing
- Branching
- Pull Requests
- **Pipelines**
- Architecture

- pipeline is definitive for deployment - daily stale branch job

Outline

- 1 Current challenges and their canonical solutions
- 2 Going forward
 - Team Composition
 - Committing
 - Branching
 - Pull Requests
 - Pipelines
 - Architecture